

Relatório de Desenvolvimento e Avaliação Experimental de um Servidor HTTP/1.1 Persistente em Linguagem C

Vanessa M. Otte¹, Lorhan S. Lima¹

¹Fundamentos e Avaliação de Redes de Computadores
Engenharia de Computação – Universidade Federal do Pampa (UNIPAMPA)
Bagé – RS – Brazil

{lorhanlima, vanessaotte}.aluno@unipampa.edu.br

Resumo. *Este relatório técnico apresenta a concepção, implementação e avaliação experimental de um servidor Web baseado no protocolo HTTP/1.1, correspondente ao Produto Mínimo Viável 1, implementado para testes e estudos iniciais do trabalho da disciplina. O servidor foi desenvolvido em linguagem C utilizando a biblioteca POSIX Threads (pthreads) em ambiente Linux. Seu objetivo é possibilitar conexões persistentes (Keep-Alive) e atendimento concorrente a múltiplos clientes. Para a validação experimental, foram realizadas capturas de tráfego nas camadas de transporte e aplicação por meio da ferramenta Wireshark. Os resultados obtidos comprovam a persistência da conexão TCP durante múltiplas requisições sucessivas, incluindo documento HTML, arquivo de texto, imagem e arquivo inexistente.*

1. Introdução

O protocolo HTTP é a base da comunicação de dados na Web. Em sua primeira versão, cada requisição demandava o estabelecimento e encerramento da conexão para cada pedido, o que gerava um overhead muito grande devido aos sucessivos *Three-Way Handshakes*. Na versão HTTP/1.1, isso foi mitigado pela persistência das conexões utilizando o mecanismo Keep-Alive.

O objetivo deste projeto consiste no desenvolvimento prático de um servidor Web em ambiente Linux (WSL/Ubuntu), estruturado em C com pthreads, capaz de atender clientes simultâneos de forma concorrente e reutilizar conexões abertas para o tráfego de múltiplos recursos, diminuindo o envio de requisições que não transportam carga útil. Este relatório documenta os detalhes arquiteturais do Produto Mínimo Viável 1 (MVP 1) e analisa os traces de execução coletados na interface de rede.

2. Desenvolvimento e Arquitetura do Software

A implementação do servidor foi estruturada em linguagem C utilizando a biblioteca de threads POSIX (pthreads) e a interface padrão de Sockets Berkeley em um ambiente Linux virtualizado. O software foi dividido em duas frentes complementares: o processo `main` de recepção e a função isolada de atendimento aos clientes.

2.1. Processo Principal (main)

O processo é responsável pela configuração da infraestrutura de transporte e pelo laço de aceitação contínua de conexões.

1. **Instanciação e reuso de porta.** O socket é criado via chamada `socket(AF_INET, SOCK_STREAM, 0)`. Na sequência, é chamado `setsockopt` com a opção `SO_REUSEADDR`, evitando que reinicializações do servidor sejam bloqueadas pelo estado de espera de porta `TIME_WAIT` do kernel.

```
1 int main() {
2     int idServidor = socket(AF_INET, SOCK_STREAM, 0);
3     int opcao = 1;
4     setsockopt(idServidor, SOL_SOCKET, SO_REUSEADDR,
5               &opcao, sizeof(opcao));
6     ...
7 }
```

2. **Vinculação e escuta.** Através da estrutura `struct sockaddr_in`, o socket é associado à constante `INADDR_ANY`, que permite ler qualquer endereço de entrada, e à porta 8080, convertida com `htons`. O servidor passa ao modo passivo através de `listen(idServidor, 10)`, reservando uma fila de pendência no sistema operacional.

```
1 struct sockaddr_in endereco;
2 endereco.sin_family = AF_INET;
3 endereco.sin_addr.s_addr = INADDR_ANY;
4 endereco.sin_port = htons(PORTA);
5
6 if (bind(idServidor, (struct sockaddr *)&endereco,
7         sizeof(endereco)) < 0) {
8     perror("Erro no bind! Porta ja em uso");
9     exit(1);
10 }
11
12 listen(idServidor, 10);
13 printf("Servidor aguardando conexoes na porta %d...\n", PORTA);
```

3. **Desacoplamento e alocação segura na heap.** No laço de atendimento, a função `accept()` aguarda requisições. Para evitar condições de corrida, nas quais uma nova conexão poderia sobrescrever a variável do descritor antes de a thread precedente realizar a leitura, o identificador retornado é alocado dinamicamente na memória heap via `malloc(sizeof(int))`. O ponteiro exclusivo resultante é repassado como argumento para a rotina `pthread_create()`.

```
1 while (1) {
2     struct sockaddr_in enderecoCliente;
3     socklen_t tamanhoEndereco = sizeof(enderecoCliente);
4     int idCliente = accept(idServidor,
5                           (struct sockaddr *)&enderecoCliente, &tamanhoEndereco);
6     if (idCliente < 0) {
7         perror("Erro no aceite");
8         continue;
9     }
10 }
```

```

10     int *idClienteCopia = malloc(sizeof(int));
11     *idClienteCopia = idCliente;
12     pthread_t idThread;
13     pthread_create(&idThread, NULL, atenderCliente,
14                   idClienteCopia);
15 }

```

2.2. A Rotina Concorrente da Thread (atenderCliente)

Cada conexão estabelecida opera em uma linha de execução independente sob a função `atenderCliente`.

- **Desanexamento de recursos.** A thread executa `pthread_detach(pthread_self())`. Isso informa ao sistema operacional que seus recursos de pilha e descritores devem ser coletados automaticamente ao término da rotina, mitigando o risco de threads zumbis (*zombie threads*).

```

1 void *atenderCliente(void *argumentoCliente) {
2     int *ponteiroID = (int *)argumentoCliente;
3     int idCliente = *ponteiroID;
4     free(ponteiroID);
5     pthread_detach(pthread_self());

```

- **Persistência de conexão (HTTP/1.1).** A execução entra em um laço de repetição condicionado por uma flag de controle (`manterConexao = 1`). A leitura dos dados é realizada continuamente através da chamada `recv()`. Caso o cliente encerre a transmissão ou ocorra falha de rede (`bytesLidos <= 0`), o laço é interrompido. O cabeçalho de requisição é inspecionado à procura de **Connection: close**; caso encontrado, a flag é ajustada para finalizar a conexão após o envio da resposta.

```

1 while (manterConexao == 1) {
2     char buffer[4096] = {0};
3     ssize_t bytesLidos = recv(idCliente, buffer,
4                               sizeof(buffer) - 1, 0);
5     if (bytesLidos <= 0) {
6         break;
7     }
8     ...
9     if (strstr(buffer, "Connection: close") != NULL) {
10         manterConexao = 0;
11     }
12 }

```

2.3. Parser de Requisições e Despacho de Recursos

A extração dos parâmetros da requisição e o envio dos arquivos seguem o fluxo detalhado abaixo.

1. **Extração de rotas.** Utiliza-se a função `sscanf(buffer, "%s %s", metodo, caminho)` para isolar o método HTTP e a URI requisitada. Caso a raiz (/) seja solicitada, o caminho é normalizado para `/index.html`. O prefixo relativo `.` é concatenado ao caminho por meio de `snprintf`.

```
1 sscanf(buffer, "%s %s", metodo, caminho);
2 if (strcmp(caminho, "/") == 0) {
3     strcpy(caminho, "/index.html");
4 }
5 snprintf(caminhoArquivo, sizeof(caminhoArquivo), ".%s",
6         caminho);
```

2. **Classificação.** O servidor avalia a extensão do recurso solicitado através da função `strstr()`, ajustando dinamicamente o cabeçalho `Content-Type`:

- `.html` → `text/html; charset=UTF-8`;
- `.txt` → `text/plain; charset=UTF-8`;
- `.jpg` ou `.jpeg` → `image/jpeg`;
- `.png` → `image/png`.

```
1 FILE *arquivo = fopen(caminhoArquivo, "rb");
2 char *tipoConteudo = "application/octet-stream";
3 if (strstr(caminho, ".html") != NULL) {
4     tipoConteudo = "text/html; charset=UTF-8";
5 } else if (strstr(caminho, ".jpg") != NULL ||
6     strstr(caminho, ".jpeg") != NULL) {
7     tipoConteudo = "image/jpeg";
8 } else if (strstr(caminho, ".png") != NULL) {
9     tipoConteudo = "image/png";
10 } else if (strstr(caminho, ".txt") != NULL) {
11     tipoConteudo = "text/plain; charset=UTF-8";
12 }
```

3. **Mensuração e envio em blocos (chunking).**

- **Tratamento de 404:** se o descritor retornado por `fopen()` for nulo, o servidor compõe e despacha um cabeçalho HTTP/1.1 404 Not Found.

```
1 if (arquivo == NULL) {
2     char *msg404 = "<html><body><h1>404 - Nao Encontrado"
3                 "</h1></body></html>";
4     char cabecalho404[512];
5     snprintf(cabecalho404, sizeof(cabecalho404),
6             "HTTP/1.1 404 Not Found\r\n"
7             "Content-Type: text/html; charset=UTF-8\r\n"
8             "Content-Length: %zu\r\n"
9             "Connection: %s\r\n\r\n%s",
10            strlen(msg404),
```

```

11     (manterConexao ? "keep-alive" : "close"), msg404);
12     send(idCliente, cabecalho404, strlen(cabecalho404), 0);
13 }

```

- **Envio bem-sucedido (200 OK):** quando o arquivo existe, seu tamanho total é determinado com `fseek()`, `ftell()` e `rewind()`. O cabeçalho HTTP/1.1 200 OK é formatado com o tamanho exato no campo `Content-Length`.

```

1 char cabecalho200[512];
2 snprintf(cabecalho200, sizeof(cabecalho200),
3     "HTTP/1.1 200 OK\r\n"
4     "Content-Type: %s\r\n"
5     "Content-Length: %ld\r\n"
6     "Connection: %s\r\n\r\n",
7     tipoConteudo, tamanhoArquivo,
8     (manterConexao ? "keep-alive" : "close"));
9 send(idCliente, cabecalho200, strlen(cabecalho200), 0);

```

- **Transmissão binária:** o corpo do arquivo é transferido ao socket em blocos limitados, usando buffers de 4096 bytes e as funções `fread()` e `send()`. Essa estratégia evita sobrecarga de memória RAM, permitindo o tráfego de imagens volumosas sem alocação do arquivo integral na memória do processo.

```

1 char bufferEnvio[4096];
2 size_t bytesLidosDisco;
3 while ((bytesLidosDisco = fread(bufferEnvio, 1,
4     sizeof(bufferEnvio), arquivo)) > 0) {
5     send(idCliente, bufferEnvio, bytesLidosDisco, 0);
6 }

```

3. Avaliação Experimental

A validação funcional e de desempenho foi realizada em ambiente Linux virtualizado com captura de pacotes via Wireshark na interface de loopback (lo), filtrando o tráfego na porta de operação (`tcp.port == 8080`).

3.1. Cenário Não Persistente

Em testes preliminares com a ferramenta de linha de comando `curl`, sem reuso de conexão, observou-se que cada transação HTTP exigiu um apertado de mão de três vias (SYN, SYN-ACK, ACK) e encerramento com flags [FIN, ACK], alocando portas de origem efêmeras distintas a cada operação.

3.2. Cenário Persistente (HTTP/1.1)

Para validação do reuso de conexão, foram realizadas requisições consecutivas por meio do navegador web Mozilla Firefox. A Figura 1 ilustra a captura de pacotes obtida no Wireshark.

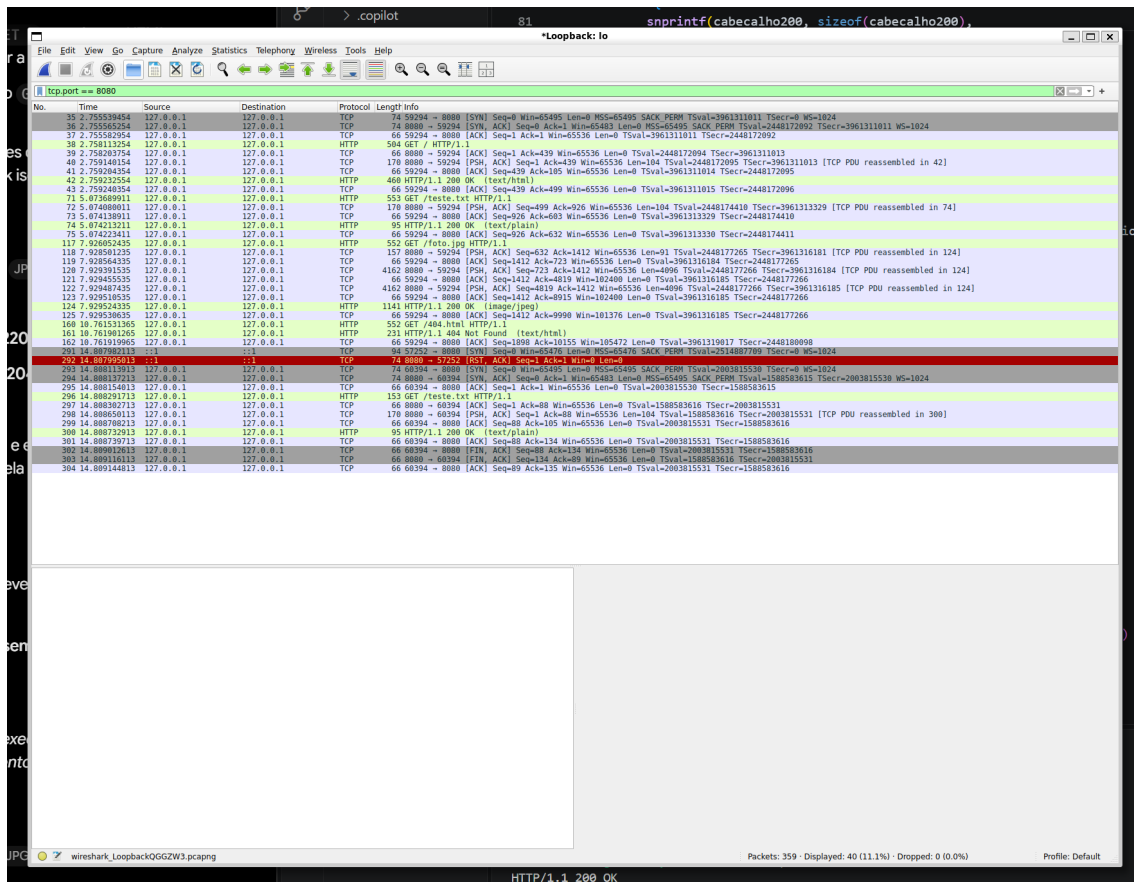


Figura 1. Traço de pacotes demonstrando persistência de conexão TCP no Wireshark.

Conforme detalhado no traço:

1. **Estabelecimento de conexão:** no pacote 35, o cliente inicia o aperto de mão TCP a partir da porta efêmera 59294 ([SYN]), completando o *Three-Way Handshake* até o pacote 37.
2. **Transferência de recursos na mesma conexão:**
 - Pacote 38: requisição GET / HTTP/1.1; o servidor responde com o código 200 OK e entrega o documento HTML (pacote 42).
 - Pacote 71: requisição GET /teste.txt HTTP/1.1 através da mesma porta 59294; a resposta é enviada com texto plano no pacote 74.
 - Pacote 117: requisição GET /foto.jpg HTTP/1.1 pela mesma sessão. O servidor transmite os dados binários da imagem segmentados em múltiplos pacotes [PSH, ACK] (pacotes 118 a 124).
 - Pacote 160: requisição GET /404.html HTTP/1.1, retornando o status de erro 404 Not Found no pacote 161.
3. **Análise do fluxo TCP:** a inspeção pela funcionalidade *Follow TCP Stream* confirmou que as quatro requisições e suas respectivas respostas transitaram sequencialmente dentro do mesmo fluxo TCP (Stream index 2), sem envio de pacotes RST ou encerramentos intermediários durante a navegação do usuário.

4. Conclusões

A implementação da primeira versão do servidor Web comprovou a eficácia do uso de conexões persistentes sob o padrão HTTP/1.1. Através dos dados coletados no Wireshark, constatou-se a redução da latência percebida e a eliminação do custo de novos handshakes TCP para o carregamento sequencial de elementos em um mesmo domínio. O desacoplamento do atendimento por threads viabilizou a resposta concorrente sem bloqueio do processo mestre. Para os próximos passos do projeto (MVP 2), a arquitetura servirá de base para a incorporação de controle de admissão e mecanismos de modulação de taxa (QoS) orientados por endereço IP.